



Midnight Credentials vs AnonCreds

Version: 0.1-draft

Status: Working comparison for design decisions

Primary Midnight source:

- `research/midnight-credentials.md`

Primary AnonCreds sources:

- <https://anoncreds.github.io/anoncreds-spec/>
- <https://anoncreds.github.io/anoncreds-spec-v2/>

Purpose

This document compares the current Midnight Credentials draft against AnonCreds.

It is not a standards-position paper and it is not trying to declare one model "better" in the abstract.

The goal is more practical:

1. identify what Midnight already does well for Compact-native smart contracts
2. identify which AnonCreds capabilities are worth borrowing
3. identify which AnonCreds assumptions do not map cleanly to Midnight
4. keep the Midnight VC direction simple enough to implement and verify

Executive Summary

Midnight Credentials and AnonCreds solve related problems, but they optimize for different execution environments.

AnonCreds is stronger today as a privacy-preserving credential exchange model:

- hidden holder binding through a link secret

- blind issuance
- same-holder proofs across multiple credentials
- mature verifier-driven presentation request semantics
- privacy-preserving revocation support

Midnight Credentials is stronger today as a Compact-first contract model:

- strongly typed canonical representation
- direct circuit-level verification logic
- schema-specialized predicates and disclosures
- explicit alignment with Midnight DID verification methods
- easier contract-time reasoning about what is actually being checked
- the ability to turn proof verification directly into contract state changes, capability issuance, and other business outcomes

The right direction is not to copy AnonCreds object-for-object.

The right direction is:

- keep Midnight's Compact-first canonical model
- borrow AnonCreds privacy patterns where they fit
- avoid dragging in unnecessary web-stack assumptions

High-Level Comparison Table

Topic	Midnight Credentials 0.1-draft	AnonCreds	Practical assessment
Canonical representation	Compact-native typed structs and circuits are the source of truth	Protocol objects and ZKP flows are primary; implementations commonly exchange JSON- like data models	Midnight is better aligned with smart- contract execution; AnonCreds is better aligned with portable credential exchange
Main optimization	On-ledger verification and schema-specific	Privacy-preserving off-chain issuance	The two systems have different

Topic	Midnight Credentials 0.1-draft	AnonCreds	Practical assessment
target	contract logic	and presentation between issuer, holder, and verifier	centers of gravity; this should remain explicit
Schema model	SchemaRef plus package-defined, strongly typed claim layouts	Public schemas and credential definitions referenced by identifiers	Midnight is stricter and easier for circuits; AnonCreds is more ecosystem- friendly and registry-oriented
Claim encoding	Claims are modeled directly in Compact structs and commitments	Attribute encoding is part of AnonCreds processing, with verifier-side interpretation responsibilities	Midnight removes encoding ambiguity for contract use cases
Proof/ signature model	Midnight draft currently uses the Midnight Jubjub profile in a Compact-native proof model	AnonCreds uses its own blind-signature and ZKP construction	Midnight should borrow protocol properties, not copy cryptography blindly
Holder binding	Supports explicit DID- bound holder binding and secret holder- binding profiles	Uses hidden holder binding via a link secret	AnonCreds is materially better for unlinkability; Midnight is simpler for explicit DID- centric workflows
Issuance privacy	Current secret-holder flow has blinded anchors as a	Blind issuance is a first-class property	This is a real Midnight gap today

Topic	Midnight Credentials 0.1-draft	AnonCreds	Practical assessment
	prototype, but full blind issuance is not complete		
Selective disclosure	Implemented via schema-specific commitments, openings, and bounded disclosure structs	Implemented through AnonCreds presentation proofs	Both support minimization; Midnight is more explicit and schema-bound
Predicate proofs	Supported for schema-defined predicates such as age >= threshold	Supported, including inequality predicates in presentation requests	Strong conceptual alignment
Presentation request model	Typed request prototypes exist for current credential families	Mature verifier-created presentation request with requested attributes, predicates, restrictions, and revocation intervals	Midnight has adopted the right pattern, but in a narrower family-specific form
Multi-credential presentation	Not yet generalized as a first-class generic presentation bundle	First-class capability: one presentation can source data from multiple credentials	AnonCreds is ahead here
Same-holder proof across	Prototyped through reusable same-holder	Built around the shared hidden link	Midnight now has the primitive, but

Topic	Midnight Credentials 0.1-draft	AnonCreds	Practical assessment
credentials	circuits for secret and blinded secret bindings	secret	not yet the full generic presentation composition model
Pairwise pseudonyms	Prototyped via verifier-domain pseudonym derivation from a hidden holder secret	Supported through holder-secret-based privacy model	Midnight is directionally aligned here
Revocation / non-revocation	Not yet implemented in the current draft	Includes privacy-preserving non-revocation proofs	Important, but still correctly deferred for the current Midnight prototype
DID integration	First-class use of Midnight DID verification method references	Can be combined with DIDs externally, but AnonCreds itself is not DID-first	Midnight is better suited when the DID is part of the core trust and authorization model
Contract suitability	Designed to be consumed directly by Compact circuits and business contracts	Requires adaptation before it becomes a Compact-native contract model	Midnight is stronger for direct on-chain policy enforcement
Verification outcome model	Proof verification can directly gate contract circuits, mutate state, mint capabilities, emit commitments, or reject with business meaning	Primary outcome is verifier-side acceptance or rejection of a presentation	Midnight has a materially stronger path from proof to enforceable system behavior
Transport coupling	Canonical model is transport-agnostic; OID4VCI / DIDComm	Often used with Aries / DIDComm-like ecosystems	Midnight should keep transport as an outer layer, not

Topic	Midnight Credentials 0.1-draft	AnonCreds	Practical assessment
	are planned as adapters	and verifier request flows	the canonical core
Governance assumptions	Trust/governance layer is recognized but still abstract in the draft	Assumes issuer/verifier ecosystem data such as schemas, credential definitions, revocation objects	AnonCreds has a more mature surrounding ecosystem model today

Why Midnight Has Different Power

The biggest architectural difference is that Midnight is not just trying to help a verifier decide whether to trust a presentation.

Midnight is trying to make the proof itself a first-class input to business logic executed by Compact circuits.

That changes what "verification" means.

In a typical AnonCreds flow, the verifier:

1. sends a presentation request
2. receives a presentation
3. verifies the cryptographic proof
4. decides off-chain whether to grant access, approve a process, or continue a workflow

In a Midnight flow, the business contract can:

1. define the required disclosure and predicate shape
2. accept a proof as circuit input
3. verify the proof inside the contract-facing verification path
4. immediately turn successful verification into a deterministic system effect

That deterministic effect can be:

- allowing the caller to execute a protected action
- mutating smart-contract state
- issuing a capability commitment or token for later use
- recording a privacy-preserving eligibility outcome
- rejecting the call with a business-meaningful denial path

This is not merely "VC verification on a blockchain".

It is a different trust model in which proofs become programmable policy inputs.

Smart-Contract Power: Midnight vs AnonCreds

Dimension	Midnight	AnonCreds	Why it matters
Where verification logic lives	In contract-facing Compact circuits and application orchestration around them	Mostly in verifier software and protocol handlers	Midnight makes verification auditable as business logic, not just as wallet/verifier behavior
What happens after verification	A successful proof can directly drive contract behavior	A verifier usually performs some off-chain decision after checking the proof	Midnight closes the gap between "proof accepted" and "system enforced"
State mutation	Native fit: proof validation can guard ledger state transitions	Not part of the core AnonCreds model	Midnight is stronger for gated on-chain workflows such as voting, auctions, access registries, eligibility-controlled issuance
Capability issuance	A contract can return or record a capability commitment after proof success	Usually handled as verifier/application logic outside the credential proof	Midnight can transform proof success into reusable on-chain or off-chain authorization artifacts

Dimension	Midnight	AnonCreds	Why it matters
		model	
Failure semantics	Can model business denial states and structured outcomes around proof checks	Usually verifier accepts or rejects presentation	Midnight gives more room for domain-specific outcomes than a binary verifier decision
Policy transparency	Circuits can show exactly which predicates and disclosures are enforced	Policy often lives partly in verifier code and partly in request format	Midnight improves auditability for high-assurance domains

ZKP Power: Midnight vs AnonCreds

AnonCreds is stronger today in mature privacy-preserving credential exchange. That point should remain clear.

But Midnight has a different kind of ZKP power:

- predicates are not only presentation tricks for a verifier
- predicates can be embedded into the business contract's acceptance logic
- proof verification and policy enforcement can be kept closer together

What Midnight can do particularly well

1. Contract-native predicate enforcement

A Midnight business contract can verify a predicate such as:

- holder is over 18
- credential is not expired
- nationality is in an allowed set

- two credentials belong to the same hidden holder

and use that result immediately inside the contract flow.

That means the proof is not just informative.

It is executable policy input.

2. Strongly typed disclosure surfaces

In Midnight, the disclosure surface is explicitly modeled in Compact types.

That gives two benefits:

- the verifier contract can only ask for disclosures and predicates that the credential family actually supports
- the implementation is easier to audit because the allowed disclosure shape is bounded and explicit

AnonCreds is more flexible in request construction.

Midnight is stricter and safer for contract integration.

3. Reusable proof capabilities as packages

Midnight can package capabilities such as:

- explicit holder binding
- secret holder binding
- blinded holder binding
- same-holder proof
- verifier-domain pseudonym derivation
- schema-specific predicates

and let business contracts compose only the capabilities they need.

That capability-oriented packaging is a strong fit for Compact and for high-assurance review.

4. Better fit for deterministic system boundaries

Midnight's model is better when the proof outcome must deterministically affect:

- ledger state
- contract permissions
- capability issuance
- token flow
- regulated eligibility checks

AnonCreds is excellent when the primary problem is privacy-preserving disclosure to a verifier.

Midnight is stronger when the proof must also be the trigger for a trusted system transition.

Disclosure Power: Midnight vs AnonCreds

Both models support selective disclosure, but they do it with different strengths.

AnonCreds disclosure strengths

- mature presentation-request model
- well-established holder privacy story
- strong holder-binding privacy through hidden secrets
- better current support for privacy-preserving exchange patterns

Midnight disclosure strengths

- disclosures are modeled as explicit, typed, bounded structures
- commitments, openings, and predicates are designed with contract verification in mind
- the disclosure contract is specialization-defined, not inferred from a loose attribute map
- the same claim can support multiple modes:
 - fully hidden
 - selectively opened
 - used only for a predicate

This makes Midnight disclosure particularly powerful for business contracts that need hard guarantees about what can and cannot be revealed or proved.

A useful way to think about the difference

AnonCreds asks:

"How can the holder satisfy this verifier request with maximum privacy?"

Midnight asks:

"How can this proof be shaped so that a contract can safely and deterministically consume it?"

Those are related questions, but they are not the same question.

Where Midnight Is Stronger Than AnonCreds

Midnight is stronger when all of the following are true:

- the verifier is effectively a smart contract or a system anchored by one
- the proof result must trigger an enforceable system action
- claim layouts must be strongly typed and bounded
- disclosures and predicates must be auditable as code
- DID references are part of the trust and authorization model

Examples:

- an age-gated contract that must either allow or reject a state transition
- a voting contract that accepts only holders satisfying a credential policy
- an auction contract that requires multiple eligibility proofs before bidding
- a contract that issues a reusable capability after successful proof verification

Where AnonCreds Is Stronger Than Midnight Today

AnonCreds is stronger when all of the following are true:

- the primary goal is privacy-preserving exchange between holder and verifier
- unlinkability across relying parties is critical

- blind issuance is required now
- non-revocation proofs are required now
- multi-credential same-holder proofs must already work in a mature ecosystem

This is why the right Midnight strategy is not competitive denial.

The right strategy is selective borrowing.

Capability-by-Capability Comparison

1. Canonical data model

Midnight starts from a Compact constraint:

- contracts need bounded, strongly typed data
- claims cannot be an open-ended JSON bag
- schema-specific witness and predicate logic must compile into circuits

AnonCreds starts from a credential-exchange constraint:

- issuer, holder, and verifier exchange protocol objects
- schemas and credential definitions are referenced by IDs
- proof construction is driven by the presentation request

Conclusion:

- Midnight should keep Compact as the canonical representation
- adapters to OID4VCI, OID4VP, DIDComm, or W3C formats should stay outer-layer concerns

2. Holder binding

Midnight currently supports two families of holder binding:

- explicit holder binding through a Midnight DID verification method reference
- hidden holder binding through a secret-holder commitment and presentation witness

AnonCreds uses a hidden holder binding model centered on a link secret.

What AnonCreds gets right:

- the verifier does not need to see a stable public holder identifier
- multiple credentials can still be proven to belong to the same holder
- holder binding and verifier-visible identity are kept separate

What Midnight gets right:

- explicit DID-bound flows are easier to understand, implement, and audit
- DID-based trust relationships map directly into contract logic

Assessment:

- explicit holder binding should remain supported
- hidden holder binding should become the privacy-oriented default for advanced flows
- the spec should remain explicit that the explicit profile is pseudonymous at best, not strongly unlinkable

3. Blind issuance

AnonCreds includes blind issuance as a core privacy feature.

That matters because the issuer can sign holder-bound material without learning the holder's final secret in the clear.

Current Midnight state:

- blinded holder-binding anchors are prototyped
- full blind issuance choreography is not yet finalized
- transport and issuer obligations are not stable enough to call it complete

Assessment:

- Midnight should keep blind issuance on the roadmap
- it is worth implementing only when the issuance protocol layer is stable enough
- fake or half-blind issuance would add complexity without delivering the real privacy property

4. Selective disclosure

Midnight selective disclosure is commitment-driven and schema-specific:

- the VC stores commitments
- the VP reveals either nothing, a disclosed value plus opening, or a predicate witness
- the contract verifies the disclosure or predicate directly

AnonCreds selective disclosure is presentation-driven:

- the verifier asks for attributes and predicates
- the holder derives a proof satisfying that request

Assessment:

- both approaches support data minimization
- Midnight's approach is easier to reason about in contract code
- AnonCreds is more general-purpose for off-chain verifier ecosystems

5. Predicate proofs

Midnight already models predicates such as:

- `age >= threshold`
- other schema-specific checks over hidden claim values

AnonCreds also supports predicate proofs and is well known for using them to avoid oversharing dates and other raw attributes.

Assessment:

- this is one of the strongest alignments between the two models
- Midnight should keep predicates specialization-defined rather than introducing a generic claim language too early

6. Presentation requests

AnonCreds gives the verifier a rich presentation request:

- requested attributes

- requested predicates
- restrictions
- revocation freshness constraints
- nonce / anti-replay binding

Midnight now prototypes typed presentation requests in credential families.

That is the correct direction because it gives the holder and verifier a shared, typed contract for the proof session.

Assessment:

- Midnight should generalize the request pattern
- Midnight should not copy the exact AnonCreds JSON shape
- the important reusable idea is verifier-defined proof intent, not the transport object format

7. Same-holder proofs across multiple credentials

AnonCreds is ahead here at the model level.

Midnight has already prototyped the key primitive:

- same secret-holder witness across two credentials
- same blinded secret-holder witness across two credentials

That means Midnight already has the core building block, but not yet the fully general presentation-bundle abstraction.

Assessment:

- keep the current reusable capability circuits
- postpone any universal "bundle" abstraction until there are at least two or three real business patterns that need it
- let business contracts compose multiple credential verifications rather than forcing a premature generic super-object

8. Pairwise pseudonyms

AnonCreds shows why pairwise pseudonyms matter:

- a verifier may need stable local correlation
- the ecosystem should avoid a single global holder identifier

Midnight has already moved in this direction with verifier-domain pseudonym derivation from a hidden holder secret.

Assessment:

- this is a strong capability to keep
- the governance around verifier-domain selection matters as much as the circuit math
- pseudonym derivation belongs in privacy-oriented holder-binding profiles, not in every credential by default

9. Revocation

AnonCreds supports non-revocation proofs.

Midnight does not yet model revocation in the current draft.

Assessment:

- this is a meaningful gap
- it is still reasonable to defer it until the core issuance, presentation, and holder-binding models stabilize
- Midnight should learn from AnonCreds here, but not block the current prototype on revocation

10. Registry and ecosystem assumptions

AnonCreds expects a wider ecosystem of registries and published objects:

- schemas
- credential definitions
- revocation objects
- supporting metadata

Midnight currently stays closer to package-defined schema families and contract-aware verification.

Assessment:

- Midnight needs some governance and registry layer eventually
- that layer should likely be Midnight-specific rather than a direct clone of AnonCreds infrastructure

What Midnight Should Borrow From AnonCreds

These are the most relevant capabilities to borrow conceptually:

1. hidden holder binding as a first-class privacy tool
2. blind issuance of holder-bound material
3. verifier-scoped or pairwise pseudonyms
4. verifier-defined presentation requests
5. same-holder proofs across multiple credentials
6. privacy-preserving revocation later

What Midnight Should Not Copy Blindly

These parts should not be adopted mechanically:

1. AnonCreds object shapes as the canonical representation
2. registry assumptions that do not fit Compact package distribution
3. generic exchange objects that weaken type safety inside contracts
4. cryptographic naming and proof object structures that are specific to AnonCreds rather than Midnight

Current Midnight Positioning

If the question is:

"Which model is better for privacy-preserving credential exchange today?"

the answer is AnonCreds.

If the question is:

"Which model is better as a Compact-native contract verification substrate?"

the answer is Midnight Credentials.

If the question is:

"Which direction should Midnight take?"

the answer is:

- stay Compact-first
- keep explicit and secret holder-binding profiles
- generalize typed presentation requests
- deepen same-holder composition
- add real blind issuance later
- add revocation only after the core privacy model is stable

Decision Matrix

Capability / design choice	Keep as-is in Midnight	Borrow from AnonCreds	Avoid for now
Compact-native canonical VC/VP	Yes	No	No
Schema-specific circuits and bounded disclosures	Yes	No	No
Explicit DID-bound holder binding	Yes, as one profile	No	No
Hidden holder binding	Partial today	Yes	No
Blind issuance	No	Yes	No
Verifier-scoped pseudonyms	Partial today	Yes	No

Capability / design choice	Keep as-is in Midnight	Borrow from AnonCreds	Avoid for now
Same-holder proofs	Partial today	Yes	No
Generic multi-credential bundle format	No	Learn from requirement pattern only	Yes, until business use cases stabilize
Revocation / non-revocation proofs	No	Yes, later	No
AnonCreds wire-format compatibility as canonical model	No	No	Yes

References

- Midnight Credentials draft: `research/midnight-credentials.md`
- Midnight companion guide: `research/midnight-credentials-for-dummies.md`
- AnonCreds specification draft: <https://anoncreds.github.io/anoncreds-spec/>
- AnonCreds v2 specification overview: <https://anoncreds.github.io/anoncreds-spec-v2/>